Lab 0: Environment Gate

Duration: 25 min (self-paced, complete BEFORE Day 1 Block 1 ends)

Difficulty: Setup

Deliverable: A machine that passes every check below and shows the marker **YOU ARE READY**.

Objective

Sign in with your organization identity, install Claude Code with the native installer, and confirm your environment is green before the first lab. Nothing in Day 1 works until this does.

> ***START MARKER:*** *You have a terminal open and can type commands.*

Step 0 (OPTIONAL): Already Using Claude Code? Start Clean (5 min)

Skip this if today is your first install. If you already run Claude Code day to day, your personal setup - a global `~/.claude/CLAUDE.md`, hooks, plugins, installed skills, custom settings - will change what Claude does in these labs. Your output will not match the room's, and you will spend lab time chasing a difference that is your config, not the tool.

The fix is a throwaway profile. `CLAUDE_CONFIG_DIR` redirects **all** Claude Code configuration - credentials, settings, skills, plugins, hooks, and CLAUDE.md - to a different directory. Set it in the terminal you will use for class:

Windows (PowerShell):

```
$env:CLAUDE_CONFIG_DIR = "$HOME\.claude-teach"
```

macOS / Linux (bash/zsh):

```
export CLAUDE_CONFIG_DIR="$HOME/.claude-class"
```

Then launch `claude` and complete the sign-in flow again. **A fresh profile means a fresh sign-in** - your normal credentials live in the old directory, so the new one starts logged out. That is expected, not an error.

The variable only applies to the terminal session you set it in. Open a new terminal without it and plain `claude` uses your normal setup, untouched. Nothing is deleted.

> ***Why this is a real technique, not just a classroom trick:***

- It is how you **test risky config safely** - a new hook, an unvetted plugin, a permission rule you are not sure about. Try it in a throwaway profile before it touches your daily driver.
- It is how you **bisect "is it my config or the tool?"** - reproduce the problem in a clean profile. Still broken? It's the tool. Works fine? It's something in your config, and now you know where to look.

Expected output marker: `claude` launches and asks you to sign in - a fresh profile means a fresh sign-in. No personal skills or CLAUDE.md are loaded.

Step 1: Identity and Access Check (5 min)

Everyone in this class is on the **enterprise plan** - there are no licensing or plan-tier decisions to make today. You only need to prove you are signed in under the right identity.

1. Confirm the exact work email/identity approved for the course.
2. Sign in with that identity. Do not substitute a personal account.

3. Launch Claude Code. If access is denied, stop and contact the training coordinator with the exact error.
4. Once signed in, run `/model` to confirm the live model configuration on this machine.

Expected output marker: Claude Code opens under the approved organization identity.

Step 2: Install Claude Code - Native Installer (10 min)

> *Claude Code no longer requires Node.js or npm. The npm install path was deprecated in January 2026. Use the native installer for your OS. Windows runs natively - no WSL required.*

macOS / Linux:

```
curl -fsSL https://claude.ai/install.sh | bash
```

or Homebrew:

```
brew install --cask claude-code # or use the curl one-liner above
```

Windows (PowerShell):

```
winget install Anthropic.ClaudeCode
```

Windows (PowerShell one-liner, self-updating - preferred):

```
irm https://claude.ai/install.ps1 | iex
```

Linux package managers: `apt`, `dnf`, and `apk` packages are also available - see the install docs at code.claude.com.

Verify:

```
claude --version
```

Expected output marker: A current stable version, with **v2.1.219 or later** required for every model and orchestration feature taught in this course. Run `claude update` if needed. If you see `command not found`, close and reopen your terminal so PATH updates take effect.

Step 2b: Two installs, and why (read this - 2 min)

There are **two separate things** and people conflate them:

	What it is	You need it for
VS Code extension	Installs from the Extensions panel. Ships with its **own private copy** of Claude Code inside it.	Everything we do in the panel - all of Day 1.
Standalone CLI	The `claude` command, installed by the commands in Step 2.	Terminal commands, `claude --version`, and Day 3's headless/CI work.

Installing the extension does not put `claude` on your PATH, and installing the CLI does not install the extension. They are independent. Both write their settings to the same `~/.claude` folder.

What to do: install **both**. Do the extension first - it is one click from a UI you already know and it cannot be blocked by shell policy. Then the CLI.

If your CLI install is blocked by corporate policy: say so now, but do not panic. You can complete

all of Day 1 with the extension alone. We only strictly need the standalone CLI on Day 3.

Step 3: Git + VS Code Check (3 min)

```
git --version
```

Expected: `git version 2.x.x`

```
code --version
```

Expected: A VS Code version number. (VS Code is optional but recommended - the extension bundles the CLI. Terminal-only is fully supported.)

For the lab sample project only (not for Claude Code itself):

```
node -v
```

Expected: `v18.x` or higher. The Day 1 build project uses Node/Express. If Node is missing, install the LTS from nodejs.org.

Step 3b: VS Code Setup - Our Classroom Configuration (4 min)

Most of this team works in VS Code, so that is how we will run the class.

1. Install the **Claude Code extension** from the VS Code marketplace (search "Claude Code", publisher Anthropic).

The extension bundles the CLI - no separate install needed if you go this route.

2. Open a VS Code **integrated terminal** (`Ctrl+\) and run` claude` there.

This is the classroom setup: the CLAUDE CODE tab in the VS Code side pane. Side-by-side diffs in the editor, a rewind arrow on every message, and a mode switcher - and `Shift+Tab` cycles modes here just like the terminal. Keep an integrated terminal open too (`Ctrl+\`): it is our fallback, and Day 3's headless and CI work needs the CLI.

Why this matters (read this): two controls we use today live in the terminal, not in the extension's chat panel:

Control	VS Code panel (our setup)	Terminal
Cycle permission modes	`Shift+Tab`, or click the mode chip	`Shift+Tab`
Mode names	Manual / Edit automatically / Plan / **Auto** / Bypass permissions	Default / AcceptEdits / Plan / Auto
Effort control	Slider at the bottom of the panel	`/effort` (verify at delivery)
Rewind	Hover a message, click the rewind arrow	Double-tap `Esc`
Rewind options	Fork conversation from here / Rewind code to here / Fork conversation and rewind code	conversation / code / both
Review a change	Side-by-side diff in the editor	Inline text diff

Both surfaces do all of it - the *labels* differ, and the fourth permission mode is genuinely different (the terminal's Auto is guarded; the panel's Bypass is not). Lab instructions lead with the panel labels and give the terminal equivalent in the

margin. You are welcome to use the extension panel for reviewing diffs, which it does better than a terminal.

Check which panel you have. Press `Shift+Tab` and count the modes:

- **Five modes** (Manual, Edit automatically, Plan, **Auto**, Bypass permissions) **and an Effort slider** at the bottom - you have the full panel. This is what you want.
- **Four modes, no Auto, no Effort slider** - you are in a reduced side-pane tab. Open the full Claude Code panel instead; two of today's exercises use controls that only appear there.

Important: launching VS Code so the clean profile actually applies

`CLAUDE_CONFIG_DIR` is an environment variable, and a program only sees it if it was set **before that program started**. If you launch VS Code from the Start menu or a desktop icon, it never sees the variable and the Claude Code extension quietly loads your normal configuration - every plugin, hook, and CLAUDE.md you already had.

To get a genuinely clean VS Code session, set the variable and launch VS Code **from that same terminal**:

```
$env:CLAUDE_CONFIG_DIR = "$HOME\.claude-teach"
code .
```

```
export CLAUDE_CONFIG_DIR="$HOME/.claude-class"
code .
```

VS Code inherits the environment of the terminal that launched it, and the extension picks it up from there.

How to tell it worked: the panel should have no custom skills, no personal CLAUDE.md, and it will ask you to sign in. If it looks like your normal setup, VS Code did not inherit the variable - close it and relaunch with `code .` from the configured terminal.

(Optional, for full isolation: `code --profile "Class" .` also gives you a clean set of VS Code extensions and settings. That is separate from Claude's config - you can use both together.)

Expected output marker: `Shift+Tab` shows five modes including Auto, and an Effort slider is visible.

Step 4: Create the Course Workspace (2 min)

There is nothing to clone. You create this folder once and it is your project for all three days - every lab builds into it. Nothing gets pushed to a shared repository; the course materials you downloaded are read-only.

```
mkdir claude-code-training
cd claude-code-training
git init
```

Expected output marker: Git reports an initialized repository in the empty course workspace.

Step 4b: Native Dependency Smoke Test (3 min) - DO NOT SKIP

Lab 1A installs `better-sqlite3`, which ships prebuilt binaries for common Node versions. If your Node

version has no prebuilt binary, npm falls back to compiling from source - which needs Visual Studio build tools on Windows and fails loudly if they are missing. Find that out now, not at 10 AM.

```
mkdir dep-check && cd dep-check
npm init -y
npm install better-sqlite3
node -e "const db=require('better-sqlite3')(':memory:'); console.log('OK',
db.prepare('select 1 as x').get());"
cd .. && rm -rf dep-check
```

Expected output marker: `OK { x: 1 }`

If you see `gyp ERR! find VS` **or any compile error:** you do not have a prebuilt binary for your Node version. Two fixes, either is fine:

- Pin a newer package version: `npm install better-sqlite3@latest` and re-run the node check.
- Or tell the instructor - Lab 1A can run with Node's built-in `node:sqlite` instead.

Do not install Visual Studio build tools during class. Note your Node version (`node -v`) so the instructor can see the spread across the room.

Step 5: First Launch - Prove It Answers (5 min)

```
claude
```

1. Complete the sign-in flow **with the identity from Step 1**.
2. When the prompt appears, run:

```
/model
```

Then give it a first instruction - note this asks it to DO something, not just answer:

```
Create hello.md with one line explaining what Claude Code is, then tell me what you did.
```

It will ask permission before writing the file. Accept it. (You will get many of these today; Block 4 is where you learn to stop answering them one at a time.)

Expected output marker: `hello.md` appears in your file tree and Claude tells you what it did.

That proves four things at once: the install works, you are authenticated, a model is responding, and it can write to your workspace. That is the whole gate - no diagnostic sweep needed.

3. Confirm the default model:

```
/model
```

Expected output marker: Sonnet 5 appears in the list and is selectable. Sonnet 5 is the class default - every lab assumes it. If Sonnet 5 is not listed, flag it now; do not wait until Lab 1A.

4. Exit for now:

```
exit
```

FINISH MARKER - YOU ARE READY

If every box below is checked, you are ready for Lab 1A:

- [ ] Approved organization identity launches Claude Code successfully
- [ ] `claude --version` returns v2.1.219+ - **write your version number here:** _____

(This needs the *standalone CLI*. If you only installed the extension, check its version on the

Extensions page instead and note that here - you are still fine for Day 1.)

(Tell the instructor if you are on a notably older build. Checkpoints and rewind are recent additions - an old build will not behave like the room's.)

- [ ] `git --version` works
- [ ] Node 18+ present (for the lab project)
- [ ] Empty course workspace initialized with git
- [ ] `better-sqlite3` smoke test printed `OK { x: 1 }` - **Node version:** _____
- [ ] VS Code extension installed AND `claude` runs in the integrated terminal
- [ ] `hello.md` created by Claude (install + auth + model + write access all proven)
- [ ] `/model` shows **Sonnet 5** available and selectable
- [ ] (Optional, returning users) `CLAUDE_CONFIG_DIR` set to a clean class profile and re-authenticated

YOU ARE READY.

If Stuck

Problem	Recovery
`claude: command not found` after install	Close and reopen the terminal; on Windows, sign out/in or check PATH in System Settings
Sign-in loop / wrong account	Run `claude /logout`, then `claude` and use the Step 1 identity
Claude will not answer / errors on launch	Run `/doctor` - it is the built-in diagnostic. Read only the lines about install, PATH, and auth; ignore anything about skills, plugins, or usage. Still stuck: screenshot and post in the class channel
Corporate proxy blocks install	Ask IT to allow claude.ai and code.claude.com, or use the offline installer from your coordinator
Access denied or wrong organization	Capture the exact error and escalate to the coordinator; do not switch to a personal account
Set `CLAUDE_CONFIG_DIR` and now Claude asks me to sign in	Correct - a fresh profile is a fresh sign-in. Sign in again with the same organization identity
My results don't match the room's	You are probably on your normal profile with personal hooks/plugins/CLAUDE.md loaded. Do Step 0
Shift+Tab does nothing	Click into the Claude Code panel first so it has focus; or click the mode chip at the bottom of the panel
Want my normal setup back	Open a new terminal without `CLAUDE_CONFIG_DIR` set. Nothing was deleted
`gyp ERR! find VS` on npm install	No prebuilt binary for your Node version. `npm install better-sqlite3@latest`, or ask the instructor for the `node:sqlite` variant. Do NOT install Visual Studio in class

Problem	Recovery
`Port 3100 already in use`	Something else is listening. Use `PORT=3101` for the whole lab, or find the owner: `Get-NetTCPConnection -LocalPort 3100 -State Listen`
Git warns `LF will be replaced by CRLF`	Normal on Windows. Cosmetic, ignore it

Debrief Questions

1. What does `CLAUDE_CONFIG_DIR` actually redirect - and why does that make it a safe way to test a risky hook or plugin?
2. Why does the native installer matter compared to the old npm path?
3. Your first instruction asked Claude to *do* something rather than answer something. Why does that distinction matter?